Понятие об имитационном моделировании I

Важной сферой приложений методов Монте-Карло является имитационное моделирование сложных управляющих систем.

Отличительные черты сложной системы:

- наличие большого количества составных частей
- сложные взаимодействия между частями

На первое место выдвигаются:

- структура системы,
- организация взаимодействия между ее частями,
- осуществление функций управления.

© А.В. Зорин, ННГУ, 2021

Понятие об имитационном моделировании II

Физическая природа разворачивающихся в сложной системе процессов оказывается вторичной и может не рассматриваться.
Основной целью исследования является оценка некоторого набора показателей функционирования управляющей системы. Такая целенаправленность исследования позволяет ограничить множество подлежащих моделированию явлений.

Ввиду сложности системы аналитическое исследование и оптимизация математической модели часто недоступны. Поэтому пользуются *имитационной моделью* системы.

Имитационная модель воспроизводит последовательность явлений, наблюдаемых в процессе функционирования

© А.В. Зорин, ННГУ, 2021

Понятие об имитационном моделировании III

системы, с сохранением их логической структуры и чередования во времени.
Мы рассмотрим три подхода

- метод «черного ящика»
- моделирование дискретных событий
- нелокальное моделирование

В качестве основного подопытного объекта мы будет рассматривать системы массового обслуживания.
Основные события в СМО:

- поступление клиентов (требований),
- начало и окончание обслуживания клиентов (требований).

© А.В. Зорин, ННГУ, 2021

Метод «черного ящика» I

В рамках этого подхода полностью игнорируется действительная структура управляющей системы и протекающие в ней процессы, а имитационная модель строится по формализованному математическому описанию зависимости «выхода» системы от ее «входа». Например, пусть удалось построить математическую модель в виде неоднородного процесса Маркова $\{\boldsymbol{X}(t), 0 \leqslant t < T\}$, $T \leqslant \infty$ с конечным числом состояний, условно обозначенных $\{E_0, E_1, \ldots, E_N\}$. На практике часто получается найти уравнения Колмогорова – Чепмена для вероятностей

$$p_k(t) = \mathbf{P}(\{\boldsymbol{X}(t) = E_k\}), \qquad k = 0, 1, \ldots, N$$

© А.В. Зорин, ННГУ, 2021

Метод «черного ящика» II

в виде системы линейных дифференциальных уравнений с переменными коэффициентами

$$\frac{dp_k(t)}{dt} = -\lambda_k(t) \cdot p_k(t) + \sum_{l \neq k} \lambda_{l,k}(t) \cdot p_l(t), \quad k = 0, 1, \ldots, N.$$

Здесь все

$$\lambda_k(t) > 0, \quad \lambda_{k,l}(t) \geqslant 0, \quad \lambda_k(t) = \sum_{l \neq k} \lambda_{k,l}(t).$$

Вероятностная интерпретация этих функций следующая: пусть в момент времени t процесс находится в состоянии E_k, тогда за малый промежуток времени $[t, t+h)$ изменение состояния не произойдет с вероятностью $(\lambda_k(t)h + o(h))$, либо произойдет изменение и новое

© А.В. Зорин, ННГУ, 2021

Метод «черного ящика» III

состояние будет E_l с вероятностью $(\lambda_{k,l}(t)h + o(h))$.
Найдем совместное распределение времени пребывания процесса в заданном состоянии и состояние после скачка в виде «двумерной непрерывно-дискретной плотности» $p(s, l; t, k)$. Ее смысл: вероятность того, что скачок процесса произойдет на интервале $(t + s, t + s + h)$ и новое состояние будет E_l при условии, что в момент времени t процесс находится в состоянии E_k, равна
$p(s, l; t, k)h + o(h)$.
Разобьем интервал $[t, t + s)$ на n подынтервалов точками $t < t_1 < t_2 < \ldots < t_n < t + s$, пусть $t_0 = t$ и $t_{n+1} = t + s$.
Тогда скачок произойдет когда нужно и куда нужно, если не будет скачка на каждом их интервалов $[t_{j-1}, t_j)$, $j = 1, 2,$

© А.В. Зорин, ННГУ, 2021

Метод «черного ящика» IV

$\ldots, n+1$, и на последнем произойдет нужный скачок. Вероятность этого, с учетом марковского свойства, есть

$$\prod_{j=0}^{n}(1-\lambda_k(t_j)(t_{j+1}-t_j)+o(\Delta t_{j+1}))(\lambda_{k,l}(t+s)h+o(h)) \approx$$

$$\approx e^{-\sum\limits_{j=1}^{n}\lambda_k(t_j)(t_{j+1}-t_j)}\lambda_{k,l}(t+s)h \approx \lambda_{k,l}(t+s)\cdot e^{-\int\limits_{t}^{t+s}\lambda_k(y)\,dy}\cdot h.$$

Значит, маргинальное распределение до момента скачка имеет плотность распределения

$$\lambda_k(t+s)e^{-\int_t^{t+s}\lambda_k(y)\,dy},$$

а состояние после скачка будет E_l при условии, что скачок произошел в момент $(t+s)$, с вероятностью $\lambda_{k,l}(t+s)/\lambda_k(t+s)$.

© А.В. Зорин, ННГУ, 2021

Метод «черного ящика» V

Тогда реализацию процесса $\{\boldsymbol{X}(t), t \geq 0\}$ можно получить по следующему алгоритму.

Моделирование неоднородного марковского процесса с дискретным множеством состояний

Шаг 1. Выбрать начальное состояние $\boldsymbol{X}(0) \in \{E_0, E_1, \ldots\}$ из распределения вероятностей $p_k(0)$, $k = 0, 1, \ldots$.

Шаг 2. Если текущее состояние есть E_k, сгенерировать случайную величину $\boldsymbol{\tau}$ с плотностью распределения $\lambda_k(s)e^{-\int_0^s \lambda_k(y)\,du}$ и положить время пребывания процесса в состоянии E_k равным $\boldsymbol{\tau}$.

Шаг 3. Выбрать следующее состояние из множества $\{E_0, E_1, \ldots, E_N\}$ из распределения вероятностей $\lambda_{k,l}(s)/\lambda_k(s)$, $l = 0, 1, \ldots, N$, $l \neq k$, если на Шаге 2 было $\boldsymbol{\tau} = s$. Перейти к шагу 2.

© А.В. Зорин, ННГУ, 2021

Метод «черного ящика» VI

Таким образом, не рассматриваются процессы в системе, приведшие к конкретному виду *мгновенных интенсивностей перехода* $\lambda_k(t)$, $\lambda_{k,l}(t)$.

Задача. Докажите, что для системы обслуживания с нестационарным потоком, рассмотренной на предыдущем занятии, система уравнений имеет вид:

$$
\begin{aligned}
p_0'(t) &= -\lambda(t)p_0(t) + \mu p_1(t);\\
p_k'(t) &= \lambda(t)p_{k-1}(t) - (\lambda(t) + k\mu)p_k(t) + (k+1)\mu p_{k+1}(t),\\
&\qquad k = 1, 2, \ldots, N-1;\\
p_N'(t) &= \lambda(t)p_{N-1}(t) - N\mu p_N(t).
\end{aligned}
$$

Используя это, напишите программу моделирования реализаций соответствующего неоднородного марковского процесса

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди I

Напомним здесь задачу об исследовании СМО по управлению конфликтными транспортными потоками на перекрестке с помощью светофора-автомата (из курса "Аналитические методы решения прикладных задач теории вероятностей и статистики").

На перекресток поступают два пуассоновских потока Π_1 и Π_2 с интенсивностями λ_1 и λ_2. Светофор имеет четыре циклически сменяющихся состояния: обслуживание первого потока ($\Gamma^{(1)}$), желтый свет после первого потока ($\Gamma^{(2)}$), обслуживание второго потока ($\Gamma^{(3)}$), желтый свет после второго потока ($\Gamma^{(4)}$). Длительность работы в состоянии $\Gamma^{(r)}$ равна T_r. За время T_{2j-1} обслуживания потока Π_j максимально успеют проехать ℓ_j требований, если их в очереди достаточно много.

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди II

Введем моменты $\tau_0 = 0 < \tau_1 < \tau_2 < \ldots$ смены состояния светофора, обозначим Γ_0 — состояние светофора в момент τ_0, Γ_i — состояние светофора на интервале $(\tau_{i-1}, \tau_i]$, $\varkappa_{j,i}$ число требований потока Π_j, ожидающих в очереди начала обслуживания, в момент времени τ_i; пусть $\eta_{j,i}$ — число требований потока Π_j, поступивших за интервал $(\tau_i, \tau_{i+1}]$, $\xi_{j,i}$ — максимально возможно число обслуженных требований на промежутке $(\tau_i, \tau_{i+1}]$.
Введем отображения: $v(\Gamma^{(r)}) = T_{r+1}$, $r < 4$, $v(\Gamma^{(4)}) = T_1$, $u(\Gamma^{(r)}) = \Gamma^{(r+1)}$, $r < 4$, $u(\Gamma^{(4)}) = \Gamma^{(1)}$, тогда

$$\tau_{i+1} = \tau_i + v(\Gamma_i), \quad \Gamma_{i+1} = u(\Gamma_i),$$
$$\kappa_{j,i+1} = \max\{0, \kappa_{j,i} + \eta_{j,i} - \xi_{j,i}\}, \quad i = 0, 1, \ldots.$$

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди III

При $\Gamma_i = \Gamma^{(r)}$ случайная величина $\eta_{j,i}$ имеет распределение Пуассона с параметром $\lambda_j T_{r+1}$ при $r < 4$ и $\lambda_j T_1$ при $r = 4$. Пусть $\Gamma_0 = \Gamma^{(4)}$, т.е. начало цикла. Тогда к концу зеленого света будет очередь $\kappa_{1,1}$, а на следующем цикле

$$\kappa_{1,4i+1} = \max\{0, \kappa_{1,4i-3} + \eta_{1,4i-3} + \eta_{1,4i-2} + \eta_{1,4i-1} + \eta_{1,4i} - \ell_1\}$$

При $\lambda_1(T_1 + T_2 + T_3 + T_4) - \ell_1 < 0$ существует предельное стационарное распределение

$$\lim_{i\to\infty} \mathbf{P}(\kappa_{1,4i+1} < x) = \mathbf{P}(\tilde{\kappa}_1 < x)$$

(здесь для удобства записи введена некоторая новая случайная величина $\tilde{\kappa}_1$ с таким же распределением, как и предельное распределение длины первой очереди в конце цикла). Интерес представляет вычислить предельную

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди IV

стационарную вероятность $\mathbf{P}(\tilde{\kappa}_1 \geqslant N)$ для достаточно большого порога N, т.е. «стационарную вероятность переполнения очереди».

Первый, наивный подход заключается в том, чтобы промоделировать достаточно долгую траекторию $\kappa_{1,1}$, $\kappa_{1,5}$, $\kappa_{1,9}$, ... и посчитать частоту появления события $\kappa_{1,4i+1} \geqslant N$. При этом, наша частота будет рассчитываться по *зависимым* испытаниям, и все «подводные камни» такого метода мы обсудим в следующий раз. Сейчас укажу, что такой метод непонятно как распараллелить. Вместо одной длинной траектории мы будем моделировать много траекторий, начинающихся из одного начального состояния, на фиксированное число шагов и скрестив пальцы считать, что влияние начального состояния

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди V

ослаблено и цепь находится в случайном состоянии, похожем на предельное стационарное. Вот код на языке R.

```
jam.naive <- function(lmb, ell, N=10,
            sample=1000, trans=100){
  acc <- 0.0
  for (i in 1:sample) {
    qsz <- 0
    for(j in 1:trans)
      qsz <- max( c(0, qsz-ell+rpois(1, lmb)) )
    if(qsz>=N) acc <- acc+1
    }
  return(acc/sample)
}

replicate(10, jam.naive(12.5, 14))
```

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди VI

Задание: погонять эту программу с разными параметрами и посмотреть, насколько велик разброс значений оценок. Как можно увеличить точность оценки? Моделировать выборку большего объема? Но мы знаем, что для уточнения оценки на порядок (для повторной выборки) надо число объем выборки увеличить в сто раз. А поскольку интересующая нас вероятность (при устойчивой работе светофора) ожидается небольшая, требуемое число реализаций должно быть слишком велико.
Для получения более точных оценок приходится привлекать арсенал глубоких методов теории вероятностей и теории случайных процессов. Сейчас я вам всё покажу и расскажу!

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди VII

Во-первых, есть такая теорема, что предельное распределение может быть представлено в следующем виде:

$$\tilde{\kappa}_1 = \sup\{0, \boldsymbol{X}_1, \boldsymbol{X}_1 + \boldsymbol{X}_2, \boldsymbol{X}_1 + \boldsymbol{X}_2 + \boldsymbol{X}_3, \ldots\},$$

где берется максимум нарастающих сумм независимых, одинаково распределенных величин $\boldsymbol{X}_n = \tilde{\eta}_n - \ell_1$, $\tilde{\eta}_n$ распределена по закону Пуассона с параметром $\Lambda = \lambda_1(T_1 + T_2 + T_3 + T_4)$. Поскольку $\mathsf{M}\boldsymbol{X}_n = \Lambda - \ell_1 < 0$, снос у случайного блуждания отрицательный, суммы уходят в $-\infty$, и супремум будет конечным.
Введем случайный момент достижения случайным блужданием уровня N:

$$\boldsymbol{\theta}_N = \min\{n\colon \boldsymbol{X}_1 + \boldsymbol{X}_2 + \ldots + \boldsymbol{X}_n \geqslant N\}.$$

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди VIII

Перейдем от вероятности к математическому ожиданию:

$$\mathbf{P}(\sup\{0, \boldsymbol{X}_1, \boldsymbol{X}_1 + \boldsymbol{X}_2, \boldsymbol{X}_1 + \boldsymbol{X}_2 + \boldsymbol{X}_3, \ldots\} \geqslant N) =$$
$$= \mathrm{M}I(\sup\{0, \boldsymbol{X}_1, \boldsymbol{X}_1 + \boldsymbol{X}_2, \boldsymbol{X}_1 + \boldsymbol{X}_2 + \boldsymbol{X}_3, \ldots\} \geqslant N) =$$
$$= \mathrm{M}I(\boldsymbol{\theta}_N < \infty) = \sum_{k=1}^{\infty} \mathrm{M}I(\boldsymbol{\theta}_N = k)$$

Раз встретилось математическое ожидание, для уменьшения дисперсии оценки можно попытаться воспользоваться трюками, перечисленным в лекции про вычисление интегралов. Мы применим **метод существенной выборки**. Однако при записи формул будем пользоваться не значком интеграла, а значком математического ожидания.

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди IX

Поменяем распределение у величин $\boldsymbol{X}_n$: вместо $p(k) = \mathsf{P}(\boldsymbol{X}_n = k) = \mathsf{P}(\tilde{\eta}_1 = k + \ell_1)$, $k \geqslant -\ell_1$, пусть вероятности даются другой формулой, $\tilde{p}(k)$. Обозначим $\tilde{\mathsf{M}}(\cdot)$ вероятность, вычисляемую в предположении, что $\boldsymbol{X}_1$, $\boldsymbol{X}_2$, ... имеют это новое распределение. Событие $\boldsymbol{\theta}_N = k$ зависит только от первых k величин $\boldsymbol{X}_1, \ldots, \boldsymbol{X}_k$. Поэтому,

$$\mathsf{M} I(\boldsymbol{\theta}_N = k) = \tilde{\mathsf{M}}\left(I(\boldsymbol{\theta}_N = k) \prod_{j=1}^{k} \frac{p(\boldsymbol{X}_j)}{\tilde{p}(\boldsymbol{X}_j)} \right)$$

и

$$\mathsf{M} I(\boldsymbol{\theta}_N < \infty) = \tilde{\mathsf{M}}\left(I(\boldsymbol{\theta}_N < \infty) \prod_{j=1}^{\boldsymbol{\theta}_N} \frac{p(\boldsymbol{X}_j)}{\tilde{p}(\boldsymbol{X}_j)} \right)$$

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди X

Я вам покажу *экспоненциальную замену меры* (преобразование Эшера). Введем функции $M(t) = \mathrm{M}(e^{t\boldsymbol{X}_1}) = e^{-t\ell_1}\mathrm{M}(e^{t\tilde{\eta}_1})$. Пусть

$$\tilde{p}(k) = \frac{e^{tk}}{M(t)}\, p(k),$$

Тогда

$$\prod_{j=1}^{k} \frac{p(x_j)}{\tilde{p}(x_j)} = (M(t))^k e^{-t(x_1+x_2+\ldots+x_k)}.$$

Выберем t^* такое что $M(t^*) = 1$ и снос $\tilde{\mathrm{M}}\boldsymbol{X}_n$ случайного блуждания положителен. Поэтому блуждание быстро и

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди XI

уверенно пересекает уровень N. Тогда момент $\boldsymbol{\theta}_N$ конечен с вероятностью единица и

$$\tilde{\mathrm{M}}\left(I(\boldsymbol{\theta}_N < \infty) \prod_{j=1}^{\boldsymbol{\theta}_N} \frac{p(\boldsymbol{X}_j)}{\tilde{p}(\boldsymbol{X}_j)} \right) = \tilde{\mathrm{M}}\left(e^{-t^*(\boldsymbol{X}_1 + \boldsymbol{X}_2 + \ldots + \boldsymbol{X}_{\boldsymbol{\theta}_N})} \right)$$

Следовательно, алгоритм такой: моделируем ν реализаций $\{\boldsymbol{X}_1^{(j)} + \ldots + \boldsymbol{X}_n^{(j)}, n = 1, 2, \ldots\}$, $j = 1, 2, \ldots, \nu$ до момента перескока уровня N и фиксируем величину

$$Z_j = e^{-t^*(\boldsymbol{X}_1^{(j)} + \boldsymbol{X}_2^{(j)} + \ldots + \boldsymbol{X}_{\boldsymbol{\theta}_N^{(j)}}^{(j)})}$$

и берем среднее по ν независимым (sic!) реализациям

$$\frac{1}{\nu}(Z_1 + \ldots + Z_\nu).$$

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди XII

В частности, у нас

$$M(t) = e^{-t\ell_1} \mathsf{M} e^{t\tilde{\eta}_n} = e^{\Lambda(e^t-1)-t\ell_1}.$$

Уравнение $M(t) = 1$ можно упростить до формы

$$e^t = 1 + t/\rho, \qquad \rho = \Lambda/\ell_1,$$

и решать методом последовательных приближений.
При этом,

$$\tilde{p}(k) = e^{t^* k} p(k) = \frac{(e^{t^*}\Lambda)^{k+\ell_1}}{(k+\ell_1)!} e^{-(\Lambda+t^*\ell_1)} = \frac{(e^{t^*}\Lambda)^{k+\ell_1}}{(k+\ell_1)!} e^{-(e^{t^*}\Lambda)},$$

т.е. достаточно генерировать величины $\tilde{\eta}_n$ с распределением Пуассона с параметром $e^{t^*}\Lambda > \Lambda$ вместо Λ.

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди XIII

```
slve<-function(r, eps=1.0e-6){
    t1 <- 1.0
    t2 <- log(1.0+t1/r)
    repeat{
        if(abs(t2-t1)<eps) break
        t1 <- t2
        t2 <- log(1.0+t1/r)
    }
    return(t2)
}

jam.IS <- function(lmb, ell, N=10, sample=1000){
  acc <- 0.0
  tt <- slve(lmb/ell)
  lmb1 <- exp(tt)*lmb
  for (i in 1:sample) {
```

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди XIV

```
    j <- 0
    x <- 0
    repeat{
      x <- x-ell+rpois(1, lmb1)
      j <- j+1
      if(x>=N){
        acc <- acc+exp(-tt*x)
        break
      }
    }
  }
  return(acc/sample)
}

replicate(10, jam.IS(12.5, 14))
```

© А.В. Зорин, ННГУ, 2021

Оценка вероятности переполнения очереди XV

Буквы `IS` в названии функции — от английского Importance Sampling (существенная выборка).

© А.В. Зорин, ННГУ, 2021

Метод дискретных событий I

Второй метод заключается в полном анализе изучаемой управляющей системы и выделении всех составляющих ее элементов и связей между ними. Число таких элементов для сложных систем может быть очень большим. Иногда вводят иерархию элементов системы: группируют элементы в функционально связанные блоки или *агрегаты* и отдельно проводят моделирование работы внутри каждого блока, а потом имитацию взаимодействия блоков. Для реализации этого метода вводится понятие *модельного времени*. Модельное время t определяет шкалу изменения состояний имитируемой системы. Модельное время не совпадает ни с реальным временем работы реального прообраза изучаемой управляющей системы, ни с машинным временем имитации.

© А.В. Зорин, ННГУ, 2021

Метод дискретных событий II

Как правило, при имитационном моделировании на современных ЭВМ воспроизводится дискретная временная шкала. Шаг этой шкалы зависит от выбранного способа моделирования. Первый способ создает равномерную шкалу и известен как «способ Δt». При этом фактически имитируется последовательность состояний $\boldsymbol{X}(0)$, $\boldsymbol{X}(\Delta t)$, $\ldots$, $\boldsymbol{X}(n\Delta t)$, $\ldots$, вложенная в исходный процесс. Величина шага Δt выбирается таким образом, чтобы вероятностью осуществления двух и более изменений состояния системы за время Δt можно было пренебречь. Как правило оказывается, что на большем числе тактов изменения состояния системы не происходит, поэтому фактически машинное время тратится «впустую». Поэтому второй способ связан с рассмотрением только моментов времени, в которые происходит изменение

© А.В. Зорин, ННГУ, 2021

Метод дискретных событий III

состояния системы. Такой способ называется «методом дискретных событий».

Опишем подробнее «метод дискретных событий» для имитации одной траектории функционирования сложной управляющей системы. Пусть выбрано описание для состояния системы, множество состояний управляющей системы есть S и множество возможных событий в системе есть E, $|E| = n$ — число возможных событий. Будем считать, что каждому состоянию $s \in S$ системы присвоен целочисленный *ранг* $|s| \leq n$, равный числу возможных событий в этом состоянии. Обозначим $\boldsymbol{X}(t) \in S$ — состояние системы в момент времени $t \geqslant 0$, $\boldsymbol{T}_i(t) \geq 0$ — оставшееся время с момента t до наступления i-го события, $1 \leq i \leq |\boldsymbol{X}(t)|$. Каждая из

© А.В. Зорин, ННГУ, 2021

Метод дискретных событий IV

компонент $\boldsymbol{T}_i(t)$ убывает с единичной скоростью. Пусть $\boldsymbol{X}(t) = s \in S$. В момент

$$t + \boldsymbol{T}_{\min} = \min\{\boldsymbol{T}_1(t), \boldsymbol{T}_2(t), \ldots, \boldsymbol{T}_{|\boldsymbol{s}(t)|}(t)\}$$

одна из компонент, для определенности — $\boldsymbol{T}_j(t)$, обнуляется и это интерпретируется как наступление соответствующего j-го события. Затем выбирают новое случайное состояние системы $\boldsymbol{X}(t + \boldsymbol{T}_{\max})$ с распределением $p(s'; s, j)$ соответственно наступившему событию. Пусть новым состоянием системы оказалось s''. Наступление события j-го типа может потребовать генерирования новых случайных значений некоторых из переменных $\boldsymbol{T}_i$, $i \neq j$, с новой функцией распределения $F_i(x; s, j, s'')$, зависящей от старого состояния s, нового состояния s'', типа наступившего события j и типа i

© А.В. Зорин, ННГУ, 2021

Метод дискретных событий V

события, для которого обновляется время до наступления i. Остальные компоненты $\boldsymbol{T}_i$, которые не требуют переназначения, уменьшают на величину $\boldsymbol{T}_{\min}$. Наконец, генерируют случайное время $\boldsymbol{T}_j(t + \boldsymbol{T}_{\max} + 0)$ до наступления следующего j-го события. Затем повторяется процедура выбора следующей обнуляющейся непрерывной компоненты и т. д. Итак, дискретная временная шкала в этом методе состоит из моментов времени, в которые обнуляется одна из величин $\boldsymbol{T}_1(t)$, $\boldsymbol{T}_2(t)$, ..., $\boldsymbol{T}_{|s''|}(t)$.

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий I

Приведем здесь код на языке C++, имитирующий работу перекрестка. Объявим класс, данные и методы которого 1) поддерживают внутреннее состояние объекта моделирования, и 2) делают одну элементарную операцию — обработку следующего события. В системе перекрестка мы выделим пять типов событий:

- поступление требования по первому потоку,
- поступление требования по второму потоку,
- завершение обслуживания требований по первому потоку,
- завершение обслуживания требований по второму потоку,

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий II

- смена сигнала светофора.

Конструктор требует указать все параметры системы (интенсивности входных потоков, длительности сигналов и время переезда перекрестка одной машиной).

Листинг 17: (Файл smo.h)

```
#ifndef SMO_H
#define SMO_H
#include <deque>
#include <vector>

using namespace std;

enum ev_type {AR1,AR2,SRV1,SRV2,SRV_CHNG};
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий III

```
class SMO
{
    SMO();
    double lmda[2];
    double mu[2];
    double T[4];
    double rexp(double);
    short served;
public:
    deque<double> buffer[2];
    vector<double> outstream[2];
    short server_state;
    double arr_time1, arr_time2, srv_time1,
           srv_time2,srv_chng;
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий IV

```
    double local_time;
    ev_type next_step();
    SMO(double, double, double, double,
        double, double, double, double);
};

#endif // SMO_H
```

Обратите внимание на переменную `local_time` — модельное время.
Конструктор инициализирует параметры системы (данные класса), а также устанавливает начальный сигнал светофора в желтый после второго потока и время наступления события «смена сигнала». Так что самое

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий V

первое, что сделает имитация — включит новый сигнал. Также взводятся таймеры на поступление первых событий по каждому из потоков.

Листинг 18: (Файл smo.cpp)

```
#include "smo.h"
#include <limits>
#include <cmath>
#include <iostream>

#define POSINF numeric_limits<double>::max()

using namespace std;
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий VI

```
SMO::SMO(double lmda1, double lmda2, double T1,
  double T2, double T3, double T4, double mu1,
  double mu2)
 : server_state(4), srv_time1(POSINF),
   srv_time2(POSINF), srv_chng(0)
{
    lmda[0]=lmda1; lmda[1]=lmda2;
    mu[0]=mu1; mu[1]=mu2;
    T[0]=T1; T[1]=T2; T[2]=T3; T[3]=T4;
    local_time = 0;
    arr_time1 = rexp(lmda[0]);
    arr_time2 = rexp(lmda[1]);
}
// ...
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий VII

Основная работа происходит в методе `next_event`.

Листинг 19: (Файл smo.cpp – продолжение)

```
ev_type SMO::next_step()
{
  double next_time = arr_time1,  x;
  ev_type next_type= AR1;
  if (arr_time2<next_time)
    {next_time = arr_time2; next_type=AR2;};
  if (srv_time1<next_time)
    {next_time = srv_time1; next_type=SRV1;};
  if (srv_time2<next_time)
    {next_time = srv_time2; next_type=SRV2;};
  if (srv_chng<next_time)
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий VIII

```
    {next_time = srv_chng; next_type=SRV_CHNG;};
  local_time = next_time;
  switch(next_type){
  case AR1:
    buffer[0].push_back(local_time);
    arr_time1 += rexp(lmda[0]);
    if(server_state==1 && buffer[0].size()==1 &&
       local_time+mu[0]<srv_chng)
      srv_time1=local_time+mu[0];
      break;
  case AR2:
    buffer[1].push_back(local_time);
    arr_time2 += rexp(lmda[1]);
    if(server_state==3 && buffer[1].size()==1 &&
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий IX

```
        local_time+mu[1]<srv_chng)
      srv_time2=local_time+mu[1];
    break;
   case SRV1:
    x = buffer[0].front();
    buffer[0].pop_front();
    outstream[0].push_back(local_time-x);
    served++;
    if( server_state==1  && buffer[0].size()>0 &&
          local_time+mu[0]<srv_chng)
        srv_time1 += mu[0];
    else
        srv_time1 = POSINF;
    break;
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий X

```
    case SRV2:
      x = buffer[1].front();
      buffer[1].pop_front();
      outstream[1].push_back(local_time-x);
      served++;
     if(server_state==3 && buffer[1].size()>0 &&
          local_time+mu[1]<srv_chng)
        srv_time2 += mu[1];
      else
         srv_time2 = POSINF;
      break;
    case SRV_CHNG:
#ifdef FIX_SATURATION
  unsigned q = server_state>>1;
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий XI

```
  if( server_state&1 &&
      served< floor(T[server_state-1]/mu[q])){
    short ell = floor(T[server_state-1]/mu[q]);
    for(; buffer[q].size()>0 && served<ell;
          served++)
    {
      x = buffer[q].front();
      buffer[q].pop_front();
      outstream[q].push_back(local_time-x);
    }
   }
#endif
   if(++server_state==5) server_state=1;
   srv_chng += T[server_state-1];
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий XII

```
    if(server_state==1 && buffer[0].size()>0)
    srv_time1=local_time+mu[0];
    else if(server_state==3 && buffer[1].size()>0)
      srv_time2=local_time+mu[1];
    served = 0;
    break;
  };
  return(next_type);
};
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий XIII

По проекту, сбором статистики и конкретным использованием этого класса должен заниматься другой код, внешний. В приведенном ниже примере функции `main` проводится моделирование в течение большого промежутка времени. Для проверки работы программы собирается некоторая статистика — реальные интенсивности входных потоков (должны совпадать с указанными в конструкторе), средне число требований в первой очереди в конце зеленого света, частота превышения очередью заданного порога.

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий XIV

Листинг 20: (Файл main.cpp)

```
#include <iostream>
#include "smo.h"

using namespace std;

const char* ev_names[] =
  {"arrival␣from␣Flow1", "arrival␣from␣Flow2",
    "serviced␣from␣Queue␣1",
    "serviced␣from␣Queue␣2", "signal␣changed"};

int main()
{
  // Lambda = 0.5208*24=12.5, ell1 = floor[10/0.7
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий XV

```
SMO smo(0.5208, 0.5, 10, 3, 8, 3 , 0.7, 0.57);
unsigned cycles=0, jams=0, Q1=0, N=6;
unsigned input[2];
ev_type ev;

srand( time(0) );

input[0]=input[1]=0;
for(; smo.local_time<6.0e7; )
{
  ev = smo.next_step();
  switch (ev){
    case SRV_CHNG:
    if(smo.server_state==2)
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий XVI

```
      {
        cycles++;
        Q1 += smo.buffer[0].size();
        if(smo.buffer[0].size()>=N)
        jams++;
      }
      break;
      case AR1:
        input[0]++;
        break;
      case AR2:
        input[1]++;
        break;
    }
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий XVII

```
// cout<<"Time:" << smo.local_time
//     <<"\tevent: "<< ev_names[ev]
//     <<"\tsignal: "<< smo.server_state
//   <<" | 1st queue size: "<<smo.buffer[0].size()
//   <<" 2nd queue size: "<<smo.buffer[1].size()
//    <<endl;
  }

cout<<endl<<"Total␣cycles:␣"<<cycles<<endl
    <<"Input␣intensities:␣"
    << input[0]/smo.local_time
    <<",␣"<<input[1]/smo.local_time<<endl
    <<"Output␣intensities:␣"
    << smo.outstream[0].size()/smo.local_time
```

© А.В. Зорин, ННГУ, 2021

Моделирование перекрестка методом дискретных событий XVIII

```
    <<", "
    <<smo.outstream[1].size()/smo.local_time
    <<endl
    <<"Jams per cycle = "
    << (double)jams / (double)cycles<<endl
    <<"Average size of 1st queue  "
    << (double)Q1/(double)cycles<<endl;
 return 0;
}
```

Задание: собрать проект, позапускать. Добавить функционал — обсчет средних длительностей пребывания требовани в системе на основании статистики в `smo.outstream[0]` и `smo.outstream[1]`.

© А.В. Зорин, ННГУ, 2021

«Коварство» метода дискретных событий I

Более подробно проблему сбора и обработки статистики мы рассмотрим позднее, а сейчас приведу пример, в котором возникает неожиданное расхождение между нашими ожиданиями и результатами моделирования, хотя, на первый «я все правильно сделал и я не знаю, в чем ошибка» :)

Рассмотрим моделирование методом дискретных событий системы M/M/1 с простейшим входным потоком с интенсивностью λ, экспоненциальным обслуживанием с параметром $\mu > \lambda$ и неограниченным число мест ожидания. Нас интересует предельное распределение числа $\kappa(t)$ требований в системе:

$$\lim_{t\to\infty} \mathbf{P}(\kappa(t) = x).$$

© А.В. Зорин, ННГУ, 2021

«Коварство» метода дискретных событий II

Из спецкурса по теории массового обслуживания вам должен быть известен ответ:

$$\lim_{t\to\infty} \mathbf{P}(\kappa(t) = x) = (1-\rho)\rho^x, \quad \rho = \lambda/\mu.$$

Чтобы воспользоваться готовыми средствами статистического анализа, напишем имитацию в R. Нам потребуются два таймера: время до прихода следующего требования и время до завершения обслуживания. Кроме того, будем хранить модельное время, длину очереди и тип последнего события. Поэтому состояние СМО будет храниться в векторе длины пять.

© А.В. Зорин, ННГУ, 2021

«Коварство» метода дискретных событий III

```
lmda <- 3
mu <- 10
SMO <- c(rexp(1, lmda), # [[1]] next_arrival
         Inf,           # [[2]] next_service
         0.0,           # [[3]] local_time
         0,             # [[4]] number in the queue
         NA)            # [[5]] last event type

next_step <- function(smo)
{
    if(smo[[1]]<smo[[2]]){
        smo[[4]] <- smo[[4]]+1
        smo[[3]] <- smo[[3]]+smo[[1]]
        smo[[2]] <- smo[[2]]-smo[[1]]
        smo[[1]] <- rexp(1, lmda)
        smo[[5]] <- 1 # arrival
```

© А.В. Зорин, ННГУ, 2021

«Коварство» метода дискретных событий IV

```
        if(smo[[4]]==1)
            smo[[2]] <- rexp(1, mu)
    } else {
        smo[[4]] <- smo[[4]]-1
        smo[[3]] <- smo[[3]]+smo[[2]]
        smo[[1]] <- smo[[1]]-smo[[2]]
        smo[[5]] <- 2 # service
        if(smo[[4]]>0)
            smo[[2]] <- rexp(1, mu)
        else
            smo[[2]] <- Inf
    }
    return(smo)
}
```

© А.В. Зорин, ННГУ, 2021

«Коварство» метода дискретных событий V

Предположим, что я желаю оценить стационарные вероятности для числа требований в системе. Значит, будем двигаться по шагам и каждый раз сохраняем текущую очередь и делаем много-много тактов, чтобы получить оценки «понадежней». Работаем:

```
x <- numeric(1000)
for(i in 1:length(x))
{
    SMO <- next_step(SMO)
    x[[i]]<-SMO[[4]]
    print(SMO)
}
table(x)
dgeom(0:6,1-lmda/mu)*length(x)
```

© А.В. Зорин, ННГУ, 2021

«Коварство» метода дискретных событий VI

Вот здесь советую остановиться читать и попробовать самим позапускать код в сессии R!

© А.В. Зорин, ННГУ, 2021

«Коварство» метода дискретных событий VII

Ну что, удивлены? Вроде всё правильно было написано, «по алгоритму»...
Давайте проверим. Нам нужно предельное распределение. Давайте получим распределение длины очереди в момент $t = 30$? Мы понятия не имеем пока, насколько быстро вероятности приближаются к предельным, но может, этого хватит?
Нам надо запускать имитацию из одного и того же состояния много раз, чтобы получить выборку. Поэтому напишем функцию инициализации состояния СМО в момент 0.

© А.В. Зорин, ННГУ, 2021

«Коварство» метода дискретных событий VIII

```
reset_smo <- function(smo){
    smo[[1]]<- rexp(1, lmda)
    smo[[2]]<- Inf
    smo[[3]]<-0.0
    smo[[4]]<-0
    smo[[5]] <- NA
    return(smo)
}
Tlim <- 30
for(i in 1:length(x))
{
    SMO <- reset_smo(SMO)
    while(SMO[[3]]<Tlim){
        y <- SMO[[4]]
        SMO <- next_step(SMO)
    }
```

© А.В. Зорин, ННГУ, 2021

«Коварство» метода дискретных событий IX

```
    x[[i]]<-y
}
table(x)
dgeom(0:6,1-lmda/mu)*length(x)
```

Попробуйте сами!

Сейчас результаты моделирования (частоты) должно быть довольно близки к ожидаемому предельному геометрическому распределению.
Что же произошло? В действительности мы померили *два разных стационарных распределения*. Первое было распределение состояния СМО в момент изменения состояния (т.е. в специальные случайные моменты времени, связанные с течением процесса), а второе было стационарное распределение в произвольный неслучайный

© А.В. Зорин, ННГУ, 2021

«Коварство» метода дискретных событий X

момент времени, не связанный с течением процесса. Не удивительно, что это разные распределения!
На этом примере удобно продемонстрировать неожиданное свойство пуассоновского потока, которое в англоязычной литературе называется PASTA (Poisson Arrivals See Time Averages; также “макароны” по-английски). Состояние СМО в моменты поступления требований простейшего потока имеет стационарное (предельное) распределение для этой системы! Т.е. предельное распределение можно наблюдать на допредельном процессе!

© А.В. Зорин, ННГУ, 2021

«Коварство» метода дискретных событий XI

```
SMO<-reset(SMO)
for(i in 1:length(x)){
    SMO<-next_step(SMO)
    while(SMO[[5]]!=1) SMO<-next_step(SMO);
    x[[i]]<-SMO[[4]]
}
table(x); dgeom(0:6,1-lmda/mu)*length(x)
```

© А.В. Зорин, ННГУ, 2021

Нелокальное моделирование I

Иногда в функционировании системы обслуживания удается выделить последовательность моментов времени, так что можно найти функциональные соотношения между состояниями разных частей системы в два последовательных моменты, и удается найти условные распределения для числа поступивших требований и для максимально возможного числа поступивших требований. А задать вероятностные свойства для произвольного момента времени между моментами времени из этой последовательности не удается. В этом случае приходится моделировать изменения состояния системы только в моменты этой дискретной последовательности.

Рассмотрим снова пример с перекрестком. Одно из важных свойств простейшего потока — отсутствие

© А.В. Зорин, ННГУ, 2021

Нелокальное моделирование II

последействия. Т.е., моменты поступления требований после окончания данного сигнала светофора не зависят от числа и моментов поступления требований за время этого сигнала. Во-вторых, для простейшего потока условное совместное распределение моментов поступления вызовов при условии, что всего их n штук, есть распределение упорядоченных по возрастанию n независимых равномерно распределенных на интервале случайных величин. Ниже приведем описание класса и реализацию метода перехода к следующему состоянию.

© А.В. Зорин, ННГУ, 2021

Нелокальное моделирование III

```
class SMO
{
    double lmda[2];
    double mu[2];
    double T[4];
    double runif(double, double);
    double rpois(double);
    SMO();
    RngStream *rng;
public:
    deque<double> buffer[2];
    vector<double> outstream[2];
    short server_state;
    double local_time;
    void next_step();
    SMO(double, double, double, double,
```

© А.В. Зорин, ННГУ, 2021

Нелокальное моделирование IV

```
17          double, double, double, double);
18      ~SMO();
19  };
20
21  void SMO::next_step()
22  {
23      vector<double> eta(0);
24    server_state=server_state<4?server_state+1:1;
25      // new arrivals
26      for(int k=0; k<2; k++){
27          int n = rpois(lmda[k]*T[server_state-1]);
28          eta.clear(); eta.resize(n);
29          for(int j=0; j<n; j++)
30          {
31           eta[j] = runif(local_time,
32                         T[server_state-1]);
```

© А.В. Зорин, ННГУ, 2021

Нелокальное моделирование V

```
33          }
34          sort(eta.begin(), eta.end());
35          for(auto it=eta.begin();
36               it!=eta.end();it++)
37              buffer[k].push_back(*it);
38      }
39      // service
40      double endt = local_time+T[server_state-1];
41      if(server_state&1) // is odd?
42      {
43          int q = server_state>>1,
44              sz = buffer[q].size(),
45              ell=floor(T[server_state-1]/mu[q]);
46          double quit=local_time;
47          for(int k=0; k<min(sz,ell);k++){
48              double t = buffer[q].front();
```

© А.В. Зорин, ННГУ, 2021

Нелокальное моделирование VI

```
49                  buffer[q].pop_front();
50                  quit=(t<quit)?quit+mu[q]:t+mu[q];
51                  quit=quit>endt?endt:quit;
52                  outstream[q].push_back(quit-t);
53              }
54          }
55          local_time = endt;
56      }
```

Как видно, в этом случае мы непосредственно моделируем случайные величины Γ_i (строка 24), $\eta_{1,i}$ и $\eta_{2,i}$ (строка 27), $\kappa_{1,i}$ и $\kappa_{2,i}$ (не хранятся непосредственно, но их можно узнать с помощью `smo.buffer[j].size()`), $\xi_{1,i}$ и $\xi_{2,i}$ (строка 45), о которых говорилось в примере с перекрестком на предыдущем занятии.

© А.В. Зорин, ННГУ, 2021

Нелокальное моделирование VII

Дополнительно в строках 29–37 генерируются моменты $t_{b,i}^{(j)}$, $j = 1, 2$, $b = 1, 2, \ldots, \eta_{j,i}$. поступления вызовов, которые в итоге сохраняются в накопителях `buffer[0]` и `buffer[1]`.

Поскольку также ведется отсчет модельного времени в переменной `local_time`, мы фактически знаем для каждого пребывающего в накопителе требования, сколько оно уже провело в системе (это разность между текущим моментом и моментом поступления).

Обозначим для находящегося в j-й очереди в момент времени τ_i требования через $\zeta_{b,i}^{(j)}$, $j = 1, 2, \ldots, \kappa_{j,i}$ полное время, проведенное им в системе. Если в момент τ_i j-я очередь пуста, такие величины не определяются.

© А.В. Зорин, ННГУ, 2021

Нелокальное моделирование VIII

Тогда математически имитационная модель представляется в виде последовательности

$$\boldsymbol{X}_i = (\Gamma_i, \kappa_{1,i}, \kappa_{2,i}, \zeta_{1,i}^{(1)}, \ldots, \zeta_{\kappa_{j,i},i}^{(1)}, \zeta_{1,i}^{(2)}, \ldots, \zeta_{\kappa_{j,i},i}^{(2)},$$
$$\eta_{1,i}, \eta_{2,i}, t_{1,i}^{(1)}, \ldots, t_{\kappa_{j,i},i}^{(1)}, t_{1,i}^{(2)}, \ldots, t_{\kappa_{j,i},i}^{(2)}), \quad i = 0, 1, \ldots.$$

© А.В. Зорин, ННГУ, 2021